

计算机系统 I

第七章：汇编语言

汇编语言 (Assembly Language)

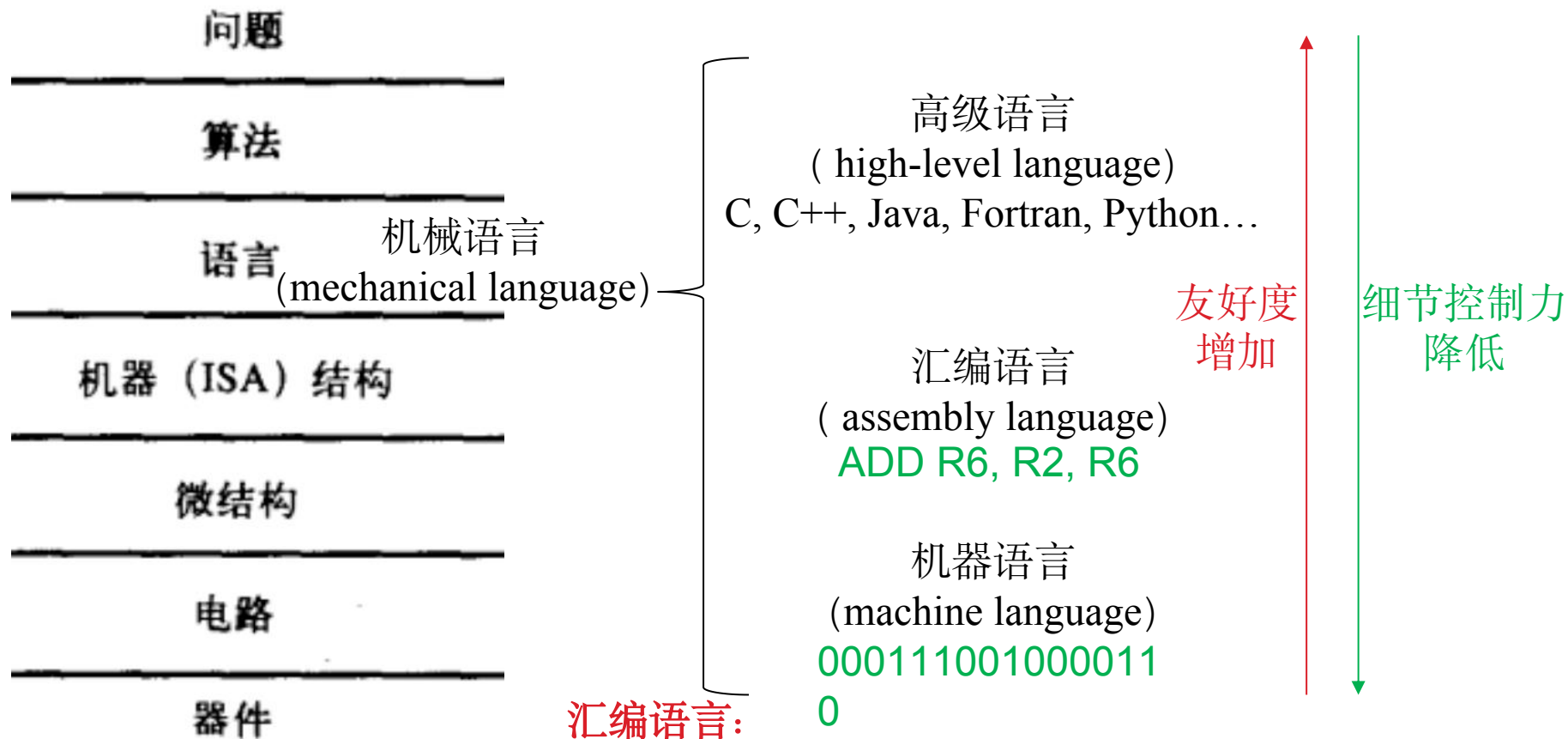


图1-6 转换层次

汇编语言:

- 为了让编程过程更加友好, 与人类自然语言不对应
- 保留细节控制能力, ISA相关
- 一条汇编语句通常对应一条机器语言指令
- 一条高级语句可能翻译成多条机器语言指令

汇编语言:具有可读性的机器语言

- ▶ 计算机的代码是由 ‘0’ , ‘1’ 组成 ...
 - **0001110010000110**
- ▶ 程序员使用机器语言编程非常困难, 采用助记符号的形式来表示每条指令将会更好, 和机器码的转换也比较容易
 - **ADD R6,R2,R6** ; *increment index reg.*

汇编器

- ▶ 汇编器 (Assembler) 是将符号语言翻译成机器指令的程序。
- ▶ 汇编器是和机器的ISA是相关的：符号与指令集定义的指令保持相应的一致性
 - 用容易记忆的符号表示操作码
 - 内存的位置用标号表示
- ▶ 为存储分配和数据的初始化提供额外的操作支持

LC-3 汇编语法

一个简单的LC-3汇编语言程序

```
;
; Program to multiply a number by the constant 6
;
; 伪操作指令
.ORIG    x3050
指令    LD      R1,  SIX
        LD      R2,  NUMBER
        AND     R3,  R3,  #0
; Clear R3.  It will
; contain the product.

; The inner loop
;
AGAIN    ADD     R3,  R3,  R2
        ADD     R1,  R1,  #-1
        BRp     AGAIN
; R1 keeps track of
; the iteration.

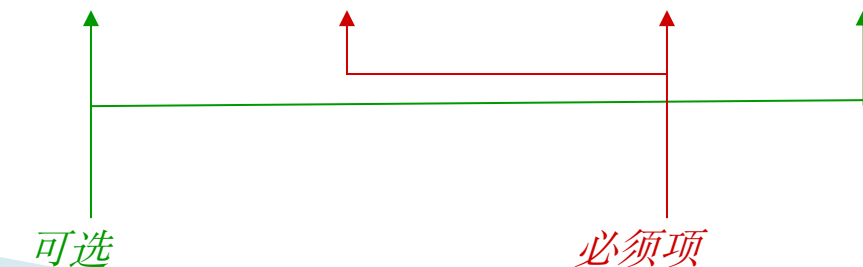
;
        HALT

;
NUMBER   .BLKW   1
SIX      .FILL   x0006
;
        .END
```

LC-3汇编语言的语法

- ▶ 程序代码的组成:
 - 可执行的机器指令
 - 伪操作指令（传递给编译器指导汇编工作，不可执行）
 - 注释
- ▶ 符号之间加入空格，不区分字母的大小写
- ▶ 注释（“;”开始），汇编器将忽略所有的注释

▶ 指令格式: LABEL OPCODE OPERANDS ; COMMENTS
 标号 操作码 操作数 ; 注释



操作码、操作数

▶ 操作码 (Opcodes)

- 和LC-3指令集定义的操作码对应的助记符号，不能再用作标号，系统专用并保留
- 具体参见Appendix A
 - ex: ADD, AND, LD, LDR, ...

▶ 操作数 (Operands)

- 寄存器数 — 寄存器 Rn, n是通用寄存器编号 ($n=0, 1, \dots, 7$)
- 立即数 — 数字用 # (十进制) or x (十六进制) 表示
- 内存数 — 标号，用符号表示的内存地址
- 用 ‘,’ 分割操作数
- 数字、操作数的顺序以及类型，**遵守机器码指令的定义**

例子

ADD R1, R1, R3

$R1 = R1 + R3$

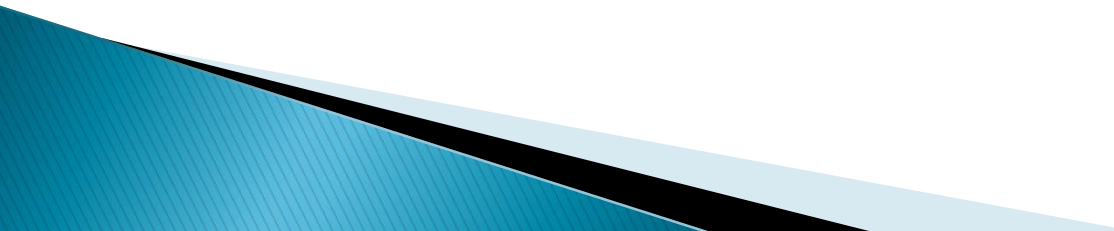
ADD R1, R1, #3

$R1 = R1 + 3$

LD R6, NUMBER

$R6 = M[NUMBER]$

BRz LOOP



数据类型

- ▶ LC-3 只有两种基本数据类型
 - 定点整数: Integer 16位补码
 - 字符: Character 16位
 - 都占用 16 bits wide (a word)
- ▶ 但实际字符只占用8 bit, 怎么存储?
 - 高位零扩展

标号 (Label)

▶ 标号: Label

- 放在每行代码的开始的地址符号，表示该行代码或数据的地址
- 符号化的内存地址。一般两种类型：
 - 分支和跳转语句的目标地址
 - 数据的存放地址
 - ex: **LOOP** ADD R1, R1, #-1
BRp **LOOP**
 - ex: LD R2, **NUMBER**

...

...

NUMBER .BLKW

1

注释 (Comment)

► 注释: Comment

- ‘;’ 之后的所有字符都是注释
- 汇编器将忽略所有的注释
- 注释用于帮助程序员理解程序和存档的需求
- 注释的技巧
 - 不要滥用注释, 比如“R1加1”, 没有提供比指令更多的信息
 - 提供更深的洞察力, 比如“指针加1指向下一个访问的数据”
 - 分隔代码片段

编译器的伪操作(Pseudo-ops)

▶ 伪操作

- 更标准的名字：汇编指令 (assembler directive)
- 不对程序产生效果，不是执行指令
- 仅供汇编器使用
- 区别于指令，以 ‘.’ 开始

<i>Opcode</i>	<i>Operand</i>	<i>Meaning</i>
.ORIG	address	指示程序起始地址
.END		指示程序在此结束，注意并不停止程序
.BLKW	<i>n</i>	分配 <i>n</i> 个字的内存单元空间
.FILL	<i>n</i>	分配一个字的内存单元空间并初始化为 <i>n</i>
.STRINGZ	<i>n</i>-character string	定义一个大小为 <i>n</i> 的字符串，占用 <i>n + 1</i> 个内存单元。第 <i>n + 1</i> 个字符为‘\0’。

伪操作: .ORIG

- ▶ 告诉编译器代码在内存中的起始地址
 - 一个程序只允许一个 .ORIG伪操作
 - PC 在程序载入时初始化为.ORIG指向的地址

▶ Example:

.ORIG x3000

LC-3的用户程序的起始地址一般设置为 x3000

- ▶ LC-3:内存分配
 - x0000 — x00FF TRAP向量表
 - x0100 — x01FF 中断向量表
 - x0200 — x2FFF 系统STACK
 - x3000 — xFDFF 用户程序区域
 - xFE00 — xFFFF 设备寄存器

伪操作: .FILL

- ▶ 在内存中定义并初始化程序变量, 可读写
- ▶ 一个程序行只允许一个定义
- ▶ 定义变量的大小总是16位的字

Examples:

flag	.FILL x0001
counter	.FILL x0002
letter	.FILL x0041
faradr	.FILL x4241

访问

LD R1,flag
;编译器会帮助产生PC相对寻址的偏移量

LDI R1,faradr
;编译器会帮助产生PC间接寻址的偏移量

伪操作: .BLKW

- ▶ 用于内存单元的分配,适用操作数一开始不确定的场合

; 保留3个未命名的内存单元

.BLKW 3

; 保留1个命名的内存单元

Bob .BLKW 1

; 保留3个命名的内存单元并全部

; 初始化为4

num .BLKW 3 #4

unamed	?
	?
	?
Bob	?
num	4
	4
	4

伪操作: .STRINGZ

- ▶ 在内存中定义1串字符串
 - 在内存中连续存放
 - 自动以 ‘\0’ 结束
 - 哨兵: “Null-terminated”
 - 每个字符高位0扩展, 占用16位

▶ Example:
hello .STRINGZ “Hello!”

▶ 访问:
LEA R0, hello
PUTS

hello	‘H’: x0048
	‘e’: x0065
	‘l’: x006c
	‘l’: x006c
	‘o’: x006f
	‘!’: x002c
	‘\0’

伪操作: .END

- ▶ 告诉编译器程序结束的地点
- ▶ 一个程序只允许一个.END
- ▶ 编译器在此停止编译，但不真正停止程序

C vs. ASM

```
int a;           // simple variable (uninitialized)
int b = 2014;    // simple initialized variable
int c[10];       // array of 10 (uninitialized)
```

```
a   .BLKW 1      ; simple variable (or .FILL 0)
b   .FILL #2014   ; simple initialized variable
c   .BLKW 10 #0   ; array of ten ints (initialized to 0)
```

```
b = a;
```

```
LD R0, a ; load from memory to a register
ST R0, b ; store from register to memory
```

```
b = a + 1;
```

```
LD R0, a      ; load from memory to a register
ADD R0, R0, #1 ; increment value
ST R0, b      ; store from register to memory
```

汇编语言程序格式

.ORIG x3000

...

your code goes here

...

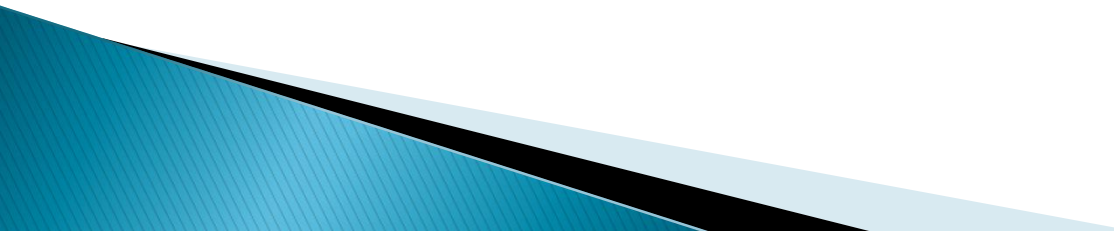
HALT

...

your memory variable definition

...

.END



编程风格

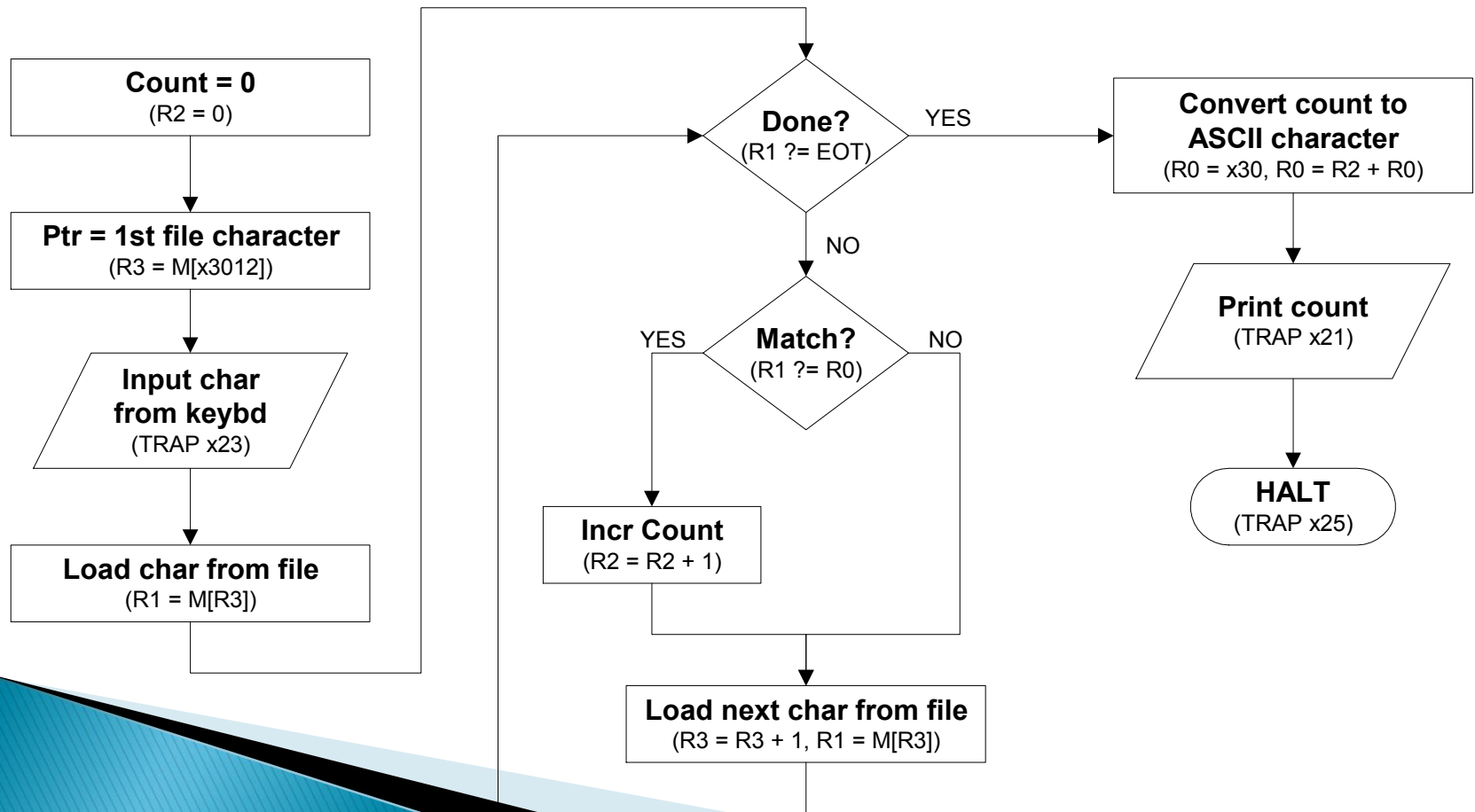
- ▶ 养成良好的编程风格，增强程序的可读性和可理解性。
- ▶ 养成写程序头的习惯，加入作者的名字，日期，以及程序的功能等。
- ▶ 标号，操作码，操作数，注释都要列对齐。（除非整行都是注释）
- ▶ 寄存器需要注释其用途。
- ▶ 为程序加入注释。

编程风格

- ▶ 符号的名字要有意义。
 - 加入适当的大写和小写字母。
 - ASCIItoBinary, InputRoutine, SaveR1
- ▶ 不同的程序段加入注释分割。
- ▶ 尽量一行一条指令。
- ▶ 较长的描述要整齐的分段。

示例程序

- 统计文件中特定字符的出现次数？还记得吗，第5章的例子



汇编语言的实现 (1/3)

程序头

```
;
; Program to count occurrences of a character in a file.
; Character to be input from the keyboard.
; Result to be displayed on the monitor.
; Program only works if no more than 9 occurrences are
found.
```

程序片段的注释
分割

程序注释, 标注寄存器的功能

```
;
;
; Initialization
;
        .ORIG    x3000
        AND      R2, R2, #0          ; R2 is counter, initially 0
        LD       R3, PTR             ; R3 points to characters
        TRAP     x23                 ; R0 gets character input
        LDR      R1, R3, #0          ; R1 gets first character

; Test character for end of file
;
TEST     ADD      R4, R1, #-4          ; Test for EOT (ASCII x04)
        BRZ      OUTPUT              ; If done, prepare the output
```

程序注释, 标注指令功能

汇编语言的实现 (2/3)

```
;
; Test character for match.  If a match, increment count.
;
        NOT        R1, R1
        ADD        R1, R1, #1 ; -R1
        ADD        R1, R1, R0 ; If match, R1 = x0000
        BRnp      GETCHAR      ; If no match, do not increment
        ADD        R2, R2, #1

;
; Get next character from file.
;
GETCHAR ADD        R3, R3, #1 ; Point to next character.
        LDR        R1, R3, #0 ; R1 gets next char to test
        BRnzp     TEST

;
; Output the count.
;
OUTPUT  LD         R0, ASCII ; Load the ASCII template
        ADD        R0, R0, R2 ; Covert binary count to ASCII
        TRAP       x21      ; ASCII code in R0 is displayed.
        HALT        ; Halt machine
```

汇编语言的实现 (2/3)

```
;  
; Storage for pointer and ASCII template  
;  
ASCII    .FILL    x0030  
PTR      .FILL    x4000  
          .END
```

Discussion: LC-3汇编程序分析

.ORIG x3000

LD R2, Zero

LD R0, M0

LD R1, M1

Loop BRz Done

ADD R2, R2, R0

ADD R1, R1, #-1

BRnzp Loop

Done ST R2, Res

HALT

Res .FILL x0000

Zero .FILL x0000

M0 .FILL x0007

M1 .FILL x0003

.END

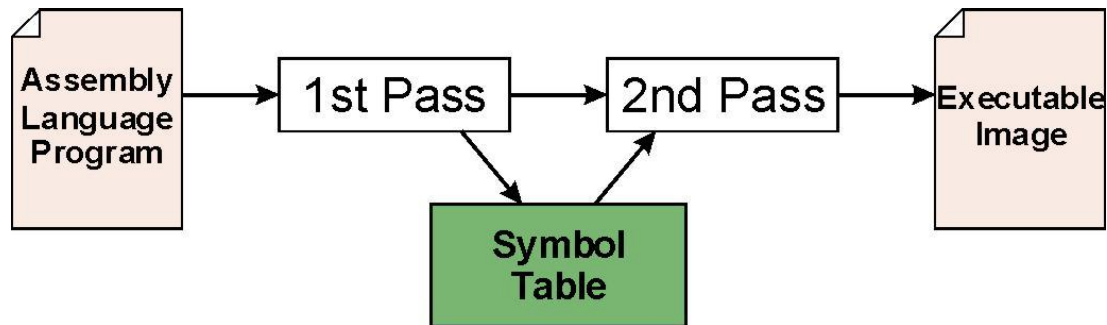
程序分析:

- 1 程序功能是什么?
- 2 最终RES的值是什么?

LC-3 汇编过程

LC-3汇编过程

- ▶ 把汇编语言源程序(.asm)转换为可执行机器代码的过程 (.obj)



- ▶ 过程：两遍扫描
- ▶ 第一遍(First Pass): 创建符号表
 - 扫描源程序文件
 - 找到所有的标号，并计算对应的地址
产生所谓的符号表 (symbol table)
- ▶ 第二遍(Second Pass): 生成机器语言程序代码
 - 利用符号表信息将指令转化为机器语言代码

第一遍： 创建符号表

1. 找到 .ORIG 伪操作
确定第一条指令的起始汇编地址
 - 初始化地址跟踪计数器 (LC: location counter), 用于记录每条当前指令的地址
2. 依次扫描源程序的每一行代码
 - 如果代码行存在标号, 将标号和指令对应的LC添加到符号表中.
 - LC+1: 如果碰到伪指令 .BLKW 或 .STRINGZ, 则LC的增量为对应分配的字数。空行不处理。
3. 碰到 .END 伪操作则停止汇编过程。

第二遍：生成机器语言程序代码

- ▶ 对每条可执行的指令，将其转换为机器语言代码
 - 如果操作数是一个标号，从符号表中查找其地址并计算
- ▶ 可能存在的问题：
 - 操作数个数或类型不对
 - ex: NOT R1, #7
 ADD R1, R2
 ADD R3, R3, NUMBER
 - 立即数的范围太大
 - ex: ADD R1, R2, #1023
 - 标号对应的地址相聚指令太远
 - 超过了PC相对寻址的范围 (-256~+255)

练习：汇编语言代码翻译成机器语言代码

```
; Program to multiply a number by  
the constant 6
```

```
                .ORIG      x3050  
                LD         R1, SIX  
                LD         R2, NUMBER  
                AND        R3, R3, #0  
  
; The inner loop  
AGAIN          ADD        R3, R3, R2  
                ADD        R1, R1, #-1  
                BRp        AGAIN  
  
                HALT  
  
NUMBER        .BLKW      1  
SIX            .FILL      x0006  
                .END
```


练习：汇编语言代码翻译成机器语言代码

第一遍扫描：创建符号表

```
; Program to multiply a number by  
the constant 6
```

```
x3050          .ORIG      x3050  
x3051          LD        R1, SIX  
x3052          LD        R2, NUMBER  
              AND        R3, R3, #0  
  
; The inner loop  
x3053 AGAIN    ADD        R3, R3, R2  
x3054          ADD        R1, R1, #-1  
x3055          BRp        AGAIN  
x3056          HALT  
  
x3057 NUMBER   .BLKW      1  
x3058 SIX      .FILL      x0006  
              .END
```

Symbol	Address
AGAIN	x3053
NUMBER	x3057
SIX	x3058

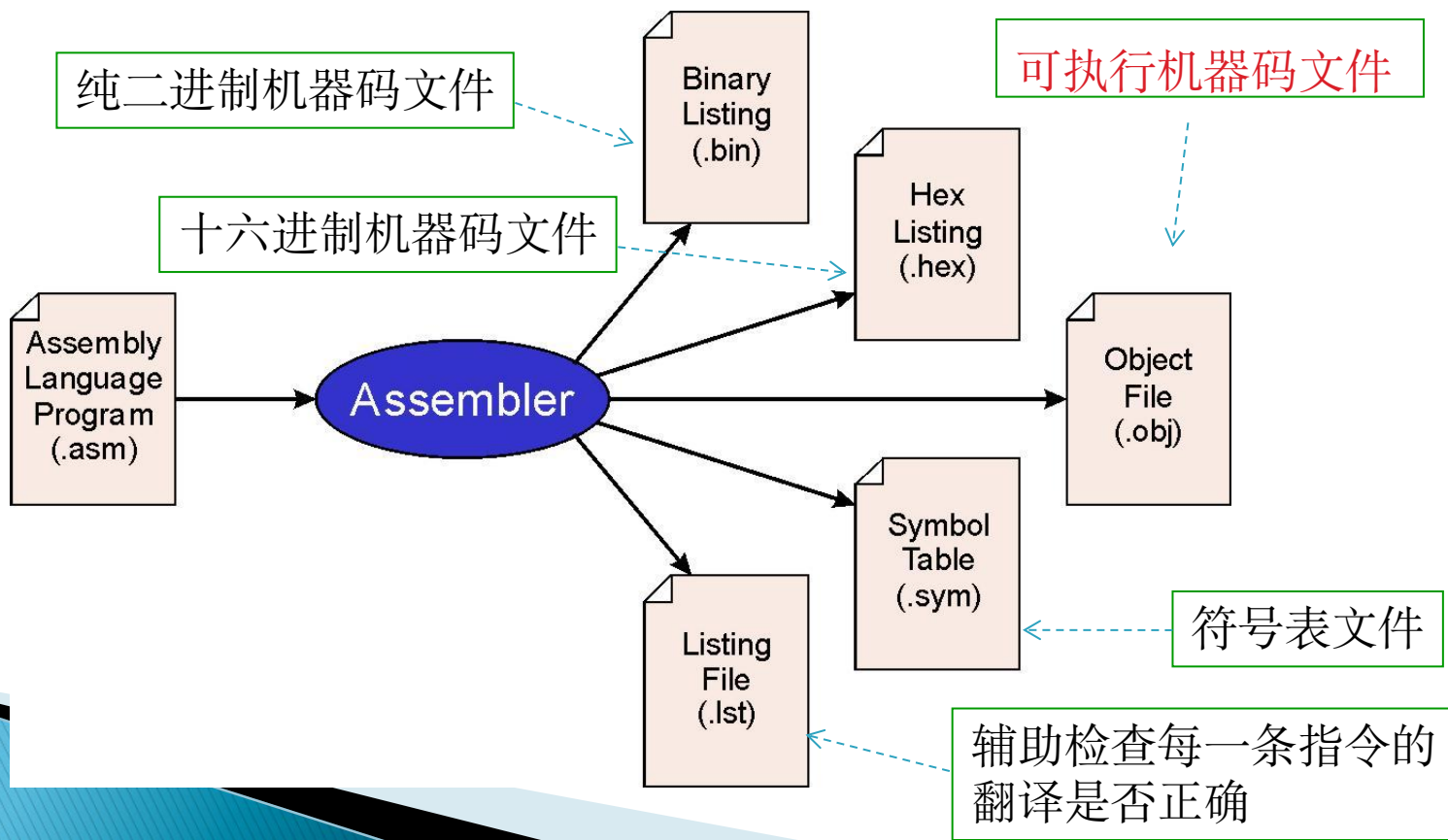
练习：汇编语言代码翻译成机器语言代码

第二遍扫描：生成机器语言程序

Address	Content	Comments
x3050	<u>0 0 1 0</u> <u>0 0 1</u> <u>0 0 0 0 0 0 1 1 1</u>	$R1 \leftarrow M[x3058 (PC+7)]$
x3051	<u>0 0 1 0</u> <u>0 1 0</u> <u>0 0 0 0 0 0 1 0 1</u>	$R2 \leftarrow M[x3057 (PC+5)]$
x3052	<u>0 1 0 1</u> <u>0 1 1</u> <u>0 1 1</u> 1 <u>0 0 0 0 0</u>	$R3 \leftarrow 0$
x3053	<u>0 0 0 1</u> <u>0 1 1</u> <u>0 1 1</u> 0 <u>0 0 0</u> <u>1 0</u>	$R3 \leftarrow R3 + R2$
x3054	<u>0 0 0 1</u> <u>0 0 1</u> <u>0 0 1</u> 1 <u>1 1 1 1 1</u>	$R1 \leftarrow R1 - 1$
x3055	<u>0 0 0 0</u> 0 0 1 <u>1 1 1 1 1 1 1 0 1</u>	$BRp\ x3053$
x3056	1 1 1 1 0 0 0 0 0 0 1 0 0 1 0 1	$HALT$
x3057		
X3058	0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0	

LC-3 汇编器

- ▶ 利用“assemble” (Unix) or LC3Edit (Windows), 产生不同的输出文件



目标文件格式

- ▶ LC-3 目标文件 (.obj) 包含以下内容
 - 起始装载地址, 随后是...
- ▶ 机器语言代码
- ▶ 例子:
 - Beginning of “count character” object file looks like this:

0011000000000000	← .ORIG x3000	第一个 word 是特殊的元数据 (加载地址)
0101010010100000	← AND R2, R2, #0	
0010011000010001	← LD R3, PTR	
1111000000100011	← TRAP x23	
.		
.		
.		

多个目标文件

- ▶ 一个目标文件可能不包含所有程序执行所需要的机器代码
 - 系统提供的库调用
 - 共享别人的成果，事先写好的子程序等
- ▶ LC-3 simulator，可以加载多个目标文件到内存，然后从指定地址执行
 - 系统调用，比如键盘输入或是显示输出等是自动装载的
 - 装入到“system memory,” 位于x3000以下
 - 用户例程应该加载到 x3000 and xFDFF
 - 每个目标文件都有起始地址，但装载时要注意不能重合覆盖

链接与加载

- ▶ **Loading (加载)** 将可执行文件拷贝到内存中.
 - 大多数的加载器能够将可执行文件重新加载到可获取的内存空间
 - 需要重新调整加载和存储跳转程序的地址
- ▶ **Linking (链接)** 将解决不同目标文件中的符号问题.
 - 假定我们在一个模块中定义的符号需要在其他的模块中使用
 - 如 `.EXTERNAL` 用于告诉汇编器该符号在其他的模块中已经定义了
 - 链接器将在多个模块的符号表中查找外部定义符号并在载入前生成机器代码

总结

- ▶ 汇编语言产生的原因：
易读、易写、自动化偏移计算等 → 提高效率
- ▶ LC-3 汇编语法：标号 操作码 操作数 ;注释
伪指令
- ▶ 汇编语言 to 机器语言过程：两遍扫描

Exercise1

7.3 如果采用字符串AND作为标识 (label)，会出什么问题？（提示：第1遍扫描会是什么情况？第2遍扫描会是什么情况？）

第1遍扫描，汇编器遇到“AND”时，不会把它放在符号表中；
第2遍扫描，当程序遇到“AND”标识时，无法找到相应的内存地址，则会报错。

7.9 伪操作.END的作用是什么？它和HALT指令之间的区别是什么？

.END 假指令告诉汇编器程序的结束位置。该指令之后出现的任何字符串都将被忽略，**不会被汇编器处理**。它与 HALT 指令在很多根本方面都有所不同：1. 这不是指令，永远无法执行。2. 因此它不会停止机器。3. 它只是一个标记，帮助汇编器知道在哪里停止汇编。

7.25 伪操作.FILL xFF004有什么问题吗？为什么？

xFF004超出了LC-3支持的16位，在汇编时就能被检测出来

Exercise2

7.1 假设一个汇编语言程序中包含了以下两条指令，汇编器将翻译后的LDI指令放在目标模块的x3025位置。问汇编过程结束后，x3025的内容是什么？

```
PLACE    .FILL    x45A7
         LDI      R3, PLACE
```

x3025 LDI R3, PLACE

PLACE=x3024

PC=x3026

LDI R3, PLACE: 1010 011 11111 1110

x3025的内容: xA7FE

Exercise3

7.13 以下程序的目的是将存放在内存A、B、C中的内容相加，并将结果存入内存。但是，代码中存在两个错误。试找出错误，并分别解释错误会在汇编时还是在运行时被检测出来？

```
Line No.
1          .ORIG x3000
2      ONE  LD R0, A

3          ADD R1, R1, R0
4      TWO  LD R0, B
5          ADD R1, R1, R0
6      THREE LD R0, C
7          ADD R1, R1, R0
8          ST R1, SUM
9          TRAP x25
10     A    .FILL x0001
11     B    .FILL x0002
12     C    .FILL x0003
13     D    .FILL x0004
14          .END
```

错误1: Line8: ST R1, SUM

SUM是未定义标签，在汇编时被检测出来

错误2: Line3: ADD R1, R1, R0

R1在使用前未初始化，相加结果可能不对，在运行时被检测出来

Exercise4

7.21 汇编以下LC-3汇编语言程序（创建符号表）：

```

                .ORIG    x3000
                AND      R0, R0, #0
                LD        R1, MASK
                ADD      R2, R0, #10
                LD        R3, PTR1
LOOP            LDR      R4, R3, #0
                AND      R4, R4, R1
                BRzP     NEXT
                ADD      R0, R0, #1
NEXT            ADD      R3, R3, #1
                ADD      R2, R2, #-1
                BRp      LOOP
                STI      R0, PTR2
MASK            .FILL    x8001
PTR1            .FILL    x4000
PTR2            .FILL    x5000
                .END
```

请问程序的目的是什么（用不多于20个字来描述）？

Exercise4

```

        .ORIG    x3000
        AND      R0, R0, #0    x3000
        LD       R1, MASK      x3001
        ADD      R2, R0, #10   x3002
        LD       R3, PTR1      x3003
LOOP     LDR      R4, R3, #0    x3004
        AND      R4, R4, R1    x3005
        BRzP     NEXT         x3006
        ADD      R0, R0, #1    x3007
NEXT     ADD      R3, R3, #1    x3008
        ADD      R2, R2, #-1   x3009
        BRp      LOOP         x300A
        STI      R0, PTR2     x300B
MASK     .FILL    x8001        x300C
PTR1     .FILL    x4000        x300D
PTR2     .FILL    x5000        x300E
        .END
```

第1遍扫描：创建符号表

Symbol	Address
LOOP	x3004
NEXT	x3008
MASK	x300C
PTR1	x300D
PTR2	x300E

第2遍扫描：生成机器语言程序

该程序计算内存地址 0x4000 至 0x4009 中负值的数量，并将结果存储在内存地址 0x5000 处。